fk_index_patch — guide

Revision: 1 **Last modified:** 2026-09-26T12:58:00Z

Source: `constitution/scripts/codegraph/fk_index_patch.py` (259 lines), plus the patch registry `runner_patches/__init__.py` (35 lines) and the shared contract `runner_patches/common.py` (59 lines). Citations below are `fk_index_patch.py:<line>` unless another file is named.

Per-patch guides (one per registered patch): fkidx1, lockfix1, datafrag1, resolve1, resolve2, mcpro1.

Overview

`fk_index_patch.py` builds a **private, version-keyed, PATCHED copy** of the CodeGraph platform package (`@colbymchenry/codegraph-linux-x64` , the bundle that holds `bin/codegraph` , `node` and `lib/dist/*.js`). The shared global install is never modified: the tool copies it (`cp -a --reflink=auto` , line 225), rewrites a handful of JavaScript files in the copy, and writes a `RUNNER_RECEIPT.json` that records exactly what was changed (lines 3-24, 62-63, 225-242). The private copy is disposable and regenerable: delete the directory and re-run (lines 10-12).

It exists because stock CodeGraph 1.6.0 has performance and safety defects that matter on a 580K-file tree (FK-cascade full scans during bulk indexing, a lock-stealing 2-minute age override, parse-pool timeouts on data-fragment headers, an O(#refs x #files) resolve query, a heap OOM in the Swift->ObjC bridge, and an MCP server that writes to the index). Each defect is fixed by one small **runner patch** module; this tool selects patches, applies them fail-closed, and packages the result.

The patch set is **selected** by `--patches` (or the registry default), and every selected patch is **applied** against a shared in-memory view of the package so a later patch sees the edits of earlier ones (`runner_patches/__init__.py:5-8`). Nothing is written to disk until every selected patch has planned successfully (lines 126-146, 201, 225-228).

Prerequisites

- `python3` , standard library only (imports lines 71-78).
- GNU `cp` with `-a` and `--reflink=auto` (line 225).
- `npm` on `PATH` , **only** when `--src` is not given: the default source is resolved with `npm root -g` (timeout 60 s, lines 104-107).
- Linux `/proc` for the "runner in use" guard (lines 149-163). Where `/proc` is absent the guard silently finds no users (line 153: the loop iterates over nothing), so the protection is off, not an error.
- The source package must be **pristine** (unpatched). Every patch except fkidx1 refuses an already-patched tree with a message saying so (see the per-patch guides); fkidx1 reports NOT_NEEDED instead (verified, below).
- It writes under `~/.cache/helix/codegraph_runner/<key>` by default (lines 207-209). **When experimenting, always pass an explicit `--dst` under a scratch directory**: the default directory is the sanctioned runner cache that live writers and MCP servers execute from. The `--force` path refuses to replace a directory whose path appears in any live process command line (lines 217-220), but do not rely on that as the only safeguard.

Usage

```
fk_index_patch.py [--src <platform package dir>] [--dst <runner dir>]
                  [--patches id,id,...] [--force] [--print-bin] [--list-patches]
```

Flag	Default	Meaning	Source
`--src DIR`	the installed platform package	Platform package to patch (read-only). Default: first child of `<npm root -g>/@colbymchenry/co-degraph/node_modules/@colbymchenry/` (sorted) that contains `lib/dist/db/in-dex.js`	lines 182, 104-116, 199
`--dst DIR`	`${HELIX_CODEGRAPH_RUNNER_DIR:-$HOME/.cache/helix/code-graph_runner}/<runner key>`	Where to build the runner. An empty `HELIX_CODEGRAPH_RUNNER_DIR` counts as unset (`or` , line 207)	lines 183, 207-209
`--patches a,b,c`	every registered patch except opt-in ones (see below)	Comma list; whitespace trimmed, empty items dropped; if nothing is left: `--patches given but empty` , exit 2	lines 184, 194-198
`--force`	off	Rebuild even when a valid re-ceipt exists (skips the reuse check)	lines 185, 212
`--print-bin`	off	On success print only the path of `bin/codegraph` instead of the JSON report	lines 186, 215, 245
`--list-patches`	off	Print `<id>\t<summary>` for the **default** patch set and exit 0; needs no `--src`	lines 187, 190-193
`-h` , `--help`		argparse help	line 181

Runner key = `<codegraph version>-<applied patch ids joined by '-'>` (line 206); ids of patches that reported NOT_NEEDED are left out. Example on stock 1.6.0 with the default set: `1.6.0-fkidx1-lockfix1-datafrag1-resolve1-resolve2`.

Examples

1. **List the default patch set** — `VERIFIED (ran: fk_index_patch.py --list-patches; saw: five tab-separated rows fkidx1, lockfix1, datafrag1, resolve1, resolve2, exit 0)`. `mcpro1` is **not** listed even though `--help` names it, because `--list-patches` uses the default set and `mcpro1` is opt-in (`runner_patches/__init__.py:18,24-25`).
2. **Build a runner in a scratch directory** (never the real cache) — `VERIFIED (ran: fk_index_patch.py --src <tiny fake package> --dst <scratch>/run --patches fkidx1; saw: JSON with "status": "CREATED", runner_key "9.9.9-fkidx1", receipt_schema 2, exit 0)`. The fake package is the fixture builder `make_pkg` from `tests/test_patch_machinery.py`; a real package copy is ~282 MB, so the scratch build used the tiny fixture. Command shape for a real package (`NOT RUN` here: it would copy 282 MB; the test suites in the Last verified section exercise that path for resolve1/resolve2/mcpro1):

   ```
   python3 scripts/codegraph/fk_index_patch.py \
     --src "$(npm root -g)/@colbymchenry/codegraph/node_modules/@colbymchenry/codegraph-linux-x64" \
     --dst /scratch/runner_test --patches resolve1
   ```

3. **Get only the launcher path** — `VERIFIED (ran: same command with --print-bin on an existing valid runner; saw: the single line <dst>/bin/codegraph, exit 0)`. This is how the callers use it: `codegraph_safe.sh` runs it with `--print-bin` (`codegraph_safe.sh:455-456`) and `codegraph_mcp.sh` with `--patches mcpro1 --print-bin` (`codegraph_mcp.sh:58`).
4. **Second run is a no-op** — `VERIFIED (ran: same build command twice; saw: "status": "REUSED" on the second run, exit 0)`.

5. **Dry-run planning against the real package without copying anything** — the CLI has no dry-run flag, but `plan_all` is importable and read-only. `VERIFIED (ran: python3 with sys.path at scripts/codegraph, import fk_index_patch; fk_index_patch.plan_all(<stock 1.6.0 dir>, None); saw: applied = ['fkidx1','lockfix1','datafrag1','resolve1','resolve2'], edited files = db/index.js, extraction/index.js, resolution/frameworks/swift-objc.js, resolution/frameworks/vue.js, resolution/index.js, utils.js; and each of the six patches, planned alone, reports APPLIED against stock 1.6.0)`. Importing the module only sets `sys.dont_write_bytecode` and edits `sys.path` (lines 80-82); it builds nothing.
6. **Refusals leave nothing behind** — `VERIFIED (ran: --patches fkidx1,mcpro1 / --patches nope / --patches ""; saw: exit 2 each with a one-line JSON {"status": "REFUSED", "reason": ...} on stderr and no <dst> directory created)`.
7. **NOT_NEEDED** — `VERIFIED (ran: --patches fkidx1 against a fake package whose queries.js no longer contains "INSERT OR REPLACE INTO nodes"; saw: JSON {"status": "NOT_NEEDED", "reason": "fkidx1: queries.js no longer uses INSERT OR REPLACE INTO nodes", ...} on stdout, exit 3, no <dst> created)`.

Outputs

stdout on success (exit 0): with `--print-bin`, one line (the launcher path); otherwise a JSON object `{"status": "CREATED"|"REUSED", "runner": <bin path>, ...receipt fields}` (lines 215, 245). On exit 3, a JSON `{"status": "NOT_NEEDED", "reason": ..., "patches": {...}}` is printed **even with `--print-bin`** (lines 202-205): callers must test the exit status before treating stdout as a path. **stderr** on exit 2: one JSON line `{"status": "REFUSED", "reason": ...}` (lines 250-255).

Files: `<dst>/` (patched copy of the package) and `<dst>/RUNNER_RECEIPT.json` (lines 229-239):

Receipt field	Meaning
`receipt_schema`	`2` (`RECEIPT_SCHEMA` , line 88)
`runner_key`	`<version>-<applied ids>` (line 206)
`patch_set`	list of applied patch ids in registry order
`codegraph_version`	version from the source `package.json` (lines 119-123)
`source`	absolute `--src` path
`source_dbjs_sha256` , `patched_dbjs_sha256`	sha256 of `lib/dist/db/index.js` before / after (`DBJS_REL` , line 89)
`patches`	per patch id: `status` (`APPLIED` or `NOT_NEEDED`), `summary` , and for APPLIED `anchors` , `files{rel: {source_sha256, patched_sha256}}` , `detail` ; for NOT_NEEDED `reason` (lines 134, 143-144)
`created_utc`	UTC timestamp `%Y-%m-%dT%H:%M:%SZ`
legacy fields	only when fkidx1 is applied: `patch_id: "fkidx1"` plus fkidx1's `kept_indexes` , `lists_before` , `lists_after` (lines 235-236) — kept for consumers of the pre-registry single-patch tool

Exit codes

Exit	Meaning	Source
0	Runner ready: CREATED now, REUSED, or `--list-patches` printed	lines 190-193, 215, 245-246
2	Fail-closed: an anchor did not occur exactly once, unknown/empty/exclusive-conflicting `--patches`, a planned edit that would be a no-op, unreadable source, a runner directory in use by a live process, a launcher that is not executable, an `OSError`, a failed `cp`, or a corrupt receipt (`JSONDecodeError`)	lines 138-140, 198, 217-220, 243-244, 250-255; `common.py:52-59` ; `runner_patches/__init__.py:26-34`
3	Every selected patch reported NOT_NEEDED (the hazard is provably absent upstream)	lines 202-205

Argparse errors use argparse's own exit 2 (usage) as well.

How the patch set is selected and applied

Selection (`runner_patches/__init__.py`)

- `REGISTRY = ["fkidx1", "lockfix1", "datafrag1", "resolve1", "resolve2", "mcpro1"]` (line 17). Order matters: patches are planned and applied in registry order on a shared view, whatever order `--patches` lists them in (line 30). Verified: `--patches resolve2,fkidx1` loads `fkidx1, resolve2` .
- **Default** (no `--patches` , `load(None)`): every registry id that is **not** opt-in (lines 24-25) — today the five writer patches. The default runner key on stock 1.6.0 is therefore `1.6.0-fkidx1-lockfix1-datafrag1-resolve1-resolve2` (`VERIFIED` , from `plan_all` , see Example 5).

- `OPT_IN = {"mcpro1"}` (line 18): selectable with `--patches mcpro1` but never part of the default set, so the read-only MCP runner cannot end up in a writer runner by omission (lines 9-13).
- `EXCLUSIVE = {"mcpro1"}` (line 19): a patch that defines a different *kind* of runner and may not be combined with any other patch; `--patches mcpro1,fkidx1` is refused with `patch 'mcpro1' is exclusive ... cannot be combined with: fkidx1` (lines 31-34; `VERIFIED`, Example 6). Consequently the runner key `<ver>-mcpro1` identifies the read-only runner exactly.
- Unknown ids: `unknown patch id(s): ... (known: ...)` (lines 26-29).

Note the difference between this tool's default and `codegraph_safe.sh`**'s** default: the wrapper passes `--patches fkidx1,resolve1` unless overridden (`codegraph_safe.sh:244`; it omits `--patches` only for `all`, lines 455-456), so the runner key it uses is `<version>-fkidx1-resolve1`, not the five-patch key.

Application (`plan_all`, lines – ; `common.py`)

1. A `FileView` over `--src` is created (`common.py:27-42`): `read(rel)` returns a pending edit if one exists, otherwise the file text.
2. For each selected module, in registry order, `mod.plan(view)` returns a `PatchPlan` (`common.py:45-59`) or raises `NotNeeded` / `Refuse`.
3. Every text replacement goes through `PatchPlan.replace_once`, which counts the anchor text and raises `Refuse` unless it occurs **exactly once** (`common.py:52-59`). `VERIFIED` for the five anchor-based patches: for every anchor of lockfix1, datafrag1, resolve1, resolve2 and mcpro1, removing it (0 occurrences) or duplicating it (2 occurrences) in a copy of the real 1.6.0 files produced `Refuse: <id>: anchor '<label>' occurs 0x|2x in <file> (expected exactly 1) — upstream shape changed, re-derive the patch`.
4. `NotNeeded` records `status: NOT_NEEDED` with the reason, skips the patch and leaves its id out of the key (lines 131-135).
5. If a plan's new text equals the old text the tool refuses (`planned edit of <rel> is a no-op — refusing`, lines 137-140): a patch that silently changes nothing would be a bluff.

6. Source and patched sha256 of every edited file go into the receipt, and the new text is layered into the view for later patches (lines 141-142).
7. Only after all plans succeed is anything copied (lines 221-228).

The six patches and how they relate

Id	Kind	Files edited (real 1.6.0)	Purpose	De-fault?
`fkidx1`	writer / bulk in-dex	`lib/dist/db/in-dex.js`	Keep the FK child-key in-dexes through the bulk-index windows so `INSERT OR REPLACE INTO nodes` under `foreign_keys=ON` never degrades into cas-cade full scans	yes
`lock-fix1`	writer / safety	`lib/dist/ utils.js`	A `codegraph.lock` is stale only when its PID is not a live codegraph pro-cess; removes the 2-minute age override	yes
`data-frag1`	writer / bulk in-dex	`lib/dist/ extraction/in-dex.js`	Store raw number-fragment C headers and NUL-con-taining files as skip rows in-stead of timing out the parse pool	yes
`re-solve1`	writer / resolve	`lib/dist/ resolution/in-dex.js` , `.../ frameworks/ vue.js`	Memoize the resolution context's file-path list per cache generation	yes
`re-solve2`	writer / resolve	`lib/dist/ resolution/ frameworks/ swift-objc.js`	Stream (never materialize) the `method` node set in the Swift->ObjC bridge	yes
`mcpro1`	**read-only MCP runner**	`db/index.js` , `utils.js` , `mcp/engine.js` , `mcp/index.js` , `bin/code-graph.js`	Make the runner incapable of mutating a project; for `serve --mcp` only	**no** (opt-in, exclus-ive)

Relations, all verified from `plan_all` on stock 1.6.0:

- The five default patches edit **disjoint files** (six files in total), so today their relative order does not change any result; the shared view and the registry order exist so a future patch can build on an earlier one.
- `fkidx1` and `mcpro1` both edit `db/index.js`, and `lockfix1` and `mcpro1` both edit `utils.js`; that overlap is a fact of the code. The reason stated in the source for exclusivity is that `mcpro1` defines a different runner kind (`runner_patches/__init__.py:10-13`).
- `resolve2` only *reads* `resolution/index.js` (it asserts the context exposes `iterateNodesByKind`, `resolve2.py:36,55-57`); `resolve1` is the patch that edits that file.
- `fkidx1` has an external oracle, `fk_cascade_probe.py` (header line 66 and `fkidx1.py:6-7`), a separate tool that `codegraph_safe.sh` runs to prove stock safe when this tool exits 3 (NOT_NEEDED) (`codegraph_safe.sh:469-473`).

Reuse, rebuild and replacement (lines –)

1. Key and destination are computed (206-210).
2. If `<dst>/RUNNER_RECEIPT.json` exists and `--force` is not given, the receipt is read and `receipt_valid` decides reuse (212-216):
   - schema-2 receipt: reuse iff `runner_key` and `patch_set` equal this invocation's, and **every edited file** in `<dst>` hashes to the freshly planned patched text (166-177);
   - legacy receipt (no `patches` key): reuse iff the applied set is exactly `["fkidx1"]`, `patch_id` is fkidx1, and both `db/index.js` hashes match (168-171).
3. Otherwise, if `<dst>` exists and any live process command line contains `<dst>/`, the tool refuses (149-163, 217-220). PIDs 1 and the tool's own PID are skipped.

4. `<dst>.partial` is removed if present, the parent directory created, the package copied, the edited files rewritten, the receipt written, the old `<dst>` deleted, and `<dst>.partial` renamed into place (221-242).
5. Last, the launcher `<dst>/bin/codegraph` must be executable, else `Refuse` (243-244). This check runs **after** the rename, so on that refusal the directory has already been swapped in.

Edge cases

- **Already-patched source.** `VERIFIED (planned each patch against the output of its own edit)` : lockfix1, datafrag1, resolve1, resolve2 and mcpro1 raise `Refuse: <id>: source is ALREADY patched — --src must be the pristine package` ; fkidx1 raises `NotNeeded: every FK child column is already served by a never-dropped index` (so a second pass reports NOT_NEEDED, not an error).
- **Upstream upgrade.** After a CodeGraph upgrade that rewrites the anchored code, the affected patch exits 2 naming the patch id and anchor, and **no runner is produced** (lines 26-32). fkidx1 is regex-based rather than anchor-based; its shape failures read e.g. `BULK_EDGE_INDEX_NAMES not found/ empty — upstream shape changed` (`VERIFIED` , list renamed in a copy).
- **NOT_NEEDED is per patch.** A patch that proves its hazard absent is skipped and dropped from the key: fkidx1 does so when `queries.js` no longer uses `INSERT OR REPLACE INTO nodes` , when `foreign_keys=ON` is gone, or when `schema.sql` has no `REFERENCES nodes(id)` (`VERIFIED` , each variant built in a copy of the real 1.6.0 files); lockfix1 does so when `STALE_TIMEOUT_MS` is gone from `utils.js` (`lockfix1.py:61-65`); datafrag1 when the upstream already records `data_fragment_skipped` and `binary_content_skipped` (`datafrag1.py:110-111`). resolve1, resolve2 and mcpro1 have no NOT_NEEDED path in their source. Exit 3 only when **all** selected patches are NOT_NEEDED (lines 202-205); `codegraph_safe.sh` reacts to exit 3 by requiring `fk_cascade_probe.py` to prove stock safe (`codegraph_safe.sh:469-473`).

- **Concurrent use.** Nothing serialises two invocations building the same `<dst>`: both would use the same `<dst>.partial` path (line 221). UNCONFIRMED: behaviour of two simultaneous builds was not run; callers such as `codegraph_safe.sh` are expected to serialise.
- **Disk.** A build copies the whole package, ~282 MB for stock 1.6.0 (`du -sh` of the installed package). `--reflink=auto` avoids the byte copy only where the filesystem supports reflinks; otherwise it is a full copy.
- **Corrupt receipt.** `VERIFIED (ran: build over a <dst> whose RUNNER_RECEIPT.json contains "{corrupt"; saw: exit 2 and {"status": "REFUSED", "reason": "JSONDecodeError: Expecting property name enclosed in double quotes: line 1 column 2 (char 1)"})`; `--force` rebuilds it (lines 212-213).
- **Bad flag.** An unrecognised option is rejected by argparse with exit 2 (`VERIFIED`, `--bogus`).

Known discrepancies and gaps (observed – –)

These are facts about the tree on the verification date, recorded so nobody relies on the tool doing more than it does.

1. **`tests/test_patch_machinery.py` is RED against this tool.** Its own header says it reproduces four review findings (P1-P4) against a tiny fake package. `VERIFIED (ran: PYTHONDONTWRITEBYTECODE=1 python3 tests/test_patch_machinery.py; saw: SUMMARY fails=9 P1b P1c P2c P2d P2e P2f P3 P4a P4b; PASS only P1a P2a P2b P4c)`. What each failing case expects that the tool does not do today:
   - **P1b/P1c** — a commented-out duplicate of, or a duplicated declaration of, `static BULK_PARSE_INDEX_NAMES = [...]` should refuse with exit 2 and build nothing; today the tool exits 0 and builds a runner (fkidx1's list parser does not assert list-anchor uniqueness).
   - **P2c/P2e** — the reuse check should verify the whole runner tree; today it verifies only the edited files, so a truncated non-edited file or a planted extra file still yields `REUSED` (lines 172-176).

- **P2d/P2f** — after the launcher is deleted, reuse should not print a path to a missing file; today `--print-bin` prints the path of a non-existent launcher (lines 212-216 never test existence).
- **P3** — expects the default patch set to be exactly `fkidx1,resolve1,resolve2`; the registry default is five patches (`fkidx1,lockfix1,datafrag1,resolve1,resolve2`).
- **P4a** — a stale `<dst>.partial` should be removed on the REUSED path; it is only removed on the rebuild path (lines 221-223).
- **P4b** — a corrupt receipt exits 2 as expected, but the error message (a `JSONDecodeError`) does not mention `--force`. The recovery itself works (P4c PASS: `--force` skips the receipt read, line 212).

UNKNOWN: whether the test is ahead of the tool (RED written first, fix pending) or the tool regressed; the constitution history in this checkout is a single squashed commit for these files, so it cannot be determined here. Reported to the conductor.

2. **In-source header is stale in four places.** (a) It lists three patches (fkidx1, lockfix1, datafrag1, lines 12-23) but the registry has six; (b) it says `--patches` defaults to "all" (line 38) whereas mcpro1 is opt-in and excluded by default; (c) its example key `1.6.0-fkidx1-lockfix1-datafrag1` (line 43) is not the default key today (`1.6.0-fkidx1-lockfix1-datafrag1-resolve1-resolve2`); (d) it cites `tests/run_runner_patch_tests.sh` "(lockfix1/datafrag1 RED/GREEN + mutations)" (line 67): **that file does not exist** (`git ls-files` finds no such path; control: the same query finds `tests/run_scope_tests.sh`). This guide follows the code, not the header.
3. **lockfix1 and datafrag1 have no behavioural test in the tree.** No file under `scripts/codegraph/tests/` contains their runtime markers (`__helixLockOwnerIsCodegraph`, `__helixDataFragClassify`, `data_fragment_skipped`, `binary_content_skipped`, `STALE_TIMEOUT_MS`; control: the same query finds `helixAllFilesMemo`). `owed_report.py` marks them "tested" because three test files merely *name* the ids (patch-set selection arguments, a registry-string mutant), and `codegraph_safe.sh:63-68` says outright that they have zero tests, which is why the wrapper does not use them by default. The two per-patch guides say what was and was not verified.

4. **Dangling link.** The guides for `codegraph_guard`, `codegraph_safe` and `disk_tripwire` link to `codegraph_runner_patches.md`; no such file exists in `docs/scripts/`.

Internal behaviour (function map)

Function	Lines	Role
`sha256`, `sha256_text`	92-101	File / text digests for receipts and reuse
`default_src`	104-116	Locate the installed platform package via `npm root -g`
`package_version`	119-123	Read `package.json` `version`
`plan_all`	126-146	Plan every selected patch on one `FileView`; NOT_NEEDED bookkeeping; no-op refusal
`runner_in_use`	149-163	PIDs whose real `/proc/<pid>/cmdline` contains `<dst>/` (never a bare `pgrep`, per the docstring)
`receipt_valid`	166-177	Reuse decision (schema-2 and legacy)
`main`	180-255	CLI, key/dst, reuse, copy, write, swap, exit-code mapping

Legacy re-exports: `FK_RE, IDX_RE, LIST_RE, TABLE_RE, compute_patch, derive_keep_set, parse_lists, patch_text` are re-exported from `fkidx1` (lines 84-85) and `default_src` is called by tests as `m.default_src()` (`tests/test_mcp_readonly.sh:62`).

Tests and mutation evidence

Test	Covers
`tests/test_patch_machinery.py`	The tool's own machinery (anchor uniqueness, reuse validation, default set, `.partial` /receipt handling); 9 of 13 checks FAIL today (above)
`tests/test_resolve1.sh`, `test_resolve2.sh`	Build a single-patch runner through this tool (`--patches resolve1` / `resolve2`) and check behaviour
`tests/test_mcp_readonly.sh`	Builds `--patches mcpro1` ; R1 receipt >= 3 anchors, R8 shape mismatch => exit 2 and no runner directory, R13 registry/opt-in/exclusive
`tests/*_mutations.sh`	Paired mutation harnesses for the above
`tests/fixtures/stub_patch_tool.sh`	Stand-in used by `codegraph_safe.sh` tests (`test_unit_safe.sh`)

Also naming this tool but not run for this guide (they may build into the real runner cache): `test_codegraph_mcp.sh` , `test_codegraph_mcp_preflight.sh` and their mutation harnesses.

Related scripts

Script / guide	Relation
fkidx1, lockfix1, datafrag1, resolve1, resolve2, mcpro1	The six registered patches
codegraph_safe	Sanctioned bulk-index entry; derives its runner from this tool (`--patches fkidx1,resolve1` by default)
`codegraph_mcp.sh`	Builds/verifies the `mcpro1` runner through this tool (`codegraph_mcp.sh:58`)
`fk_cascade_probe.py`	Oracle for fkidx1; proves stock safe when this tool exits 3
owed_report	Reports whether this tool has tests and a guide

Last verified

2026-09-26 (all runs in scratch directories under the session scratchpad; the real runner cache `~/.cache/helix/codegraph_runner` was not touched — its directory listing was identical before and after):

- `--list-patches` , `--help` , and the CLI scenarios of Examples 2-4, 6, 7 on a tiny fake package: outputs as described above.
- `plan_all` against the installed stock 1.6.0 package: each of the six patches applies alone; the default set applies as five and would produce key `1.6.0-fkidx1-lockfix1-datafrag1-resolve1-resolve2` ; fkidx1 keeps `idx_edges_target_kind` and `idx_unresolved_from_node` on this version.
- Every patched JavaScript file produced by each patch (and by the default set together) passes `node --check` with the package's bundled node (control: `node --check` on a deliberately broken file exits 1).
- `tests/test_resolve2.sh` 3/3 PASS, `test_resolve1.sh` 7/7 PASS, `test_mcp_readonly.sh` 13/13 PASS; mutation harnesses: resolve2 golden + 5/5 mutants killed, resolve1 golden + 5/5 killed, mcp_readonly 10/10 killed.

- `tests/test_patch_machinery.py` : 9 FAIL / 4 PASS (see Known discrepancies).